

Bài tập lớn – Level 3 (Bonus)

Lập trình Hàm (Functional Programming)



Viện Trí tuệ Nhân tạo, Trường Đại học Công nghệ, ĐHQGHN

Ngày 13 tháng 5 năm 2026

Mục lục

I	Giới thiệu (Introduction)	3
II	Mục tiêu học tập (Learning Objectives)	4
II.1	Hiện thực hoá bốn nguyên lý của lập trình hàm	4
II.2	Áp dụng nguyên lý trừu tượng hoá điều khiển	4
II.3	Cải tiến thiết kế đã làm ở Level 2	5
II.4	Khai thác thế mạnh của lập trình hàm	5
III	Mã nguồn cung cấp (Starter Code)	7
III.1	Tư duy thiết kế (Design Mindset)	7
III.2	Cấu trúc chương trình (Program Structure)	9
III.3	Thành phần mã nguồn	10
IV	Hướng dẫn triển khai (Implementation Guide)	13
IV.1	Mức 1 – Cơ bản (FP cú pháp)	13
IV.2	Mức 2 – Vận dụng (FP có chủ đích)	14
IV.3	Mức 3 – Nâng cao (FP thực sự mạnh)	16

V Hướng dẫn thực nghiệm (Experiment Instructions)	18
V.1 Cài đặt môi trường	18
V.2 Biên dịch chương trình	18
V.3 Chạy chương trình	19
V.4 Chạy mã chấm điểm (Grader)	19
V.5 Đo đạc cho Mức 3 nhánh B	19
V.6 Ghi nhận kết quả vào báo cáo	19
VI Yêu cầu báo cáo (Report Requirements)	22
VI.1 Định dạng và Quy định chung	22
VI.2 Cấu trúc nội dung (Outline)	22
VII Chính sách sử dụng công cụ AI (AI Usage Policy)	26
VII.1 Nguyên tắc chung	26
VII.2 Yêu cầu khai báo	26
VII.3 Những việc không được phép	26
VII.4 Mục tiêu của chính sách	27
VIII Nộp bài (Submission)	28
VIII.1 Các thành phần cần nộp	28
VIII.2 Cấu trúc thư mục	28
VIII.3 Định dạng nộp bài	29
VIII.4 Thời hạn nộp bài	29
VIII.5 Lưu ý	30

I. Giới thiệu (Introduction)

Sau khi đã hoàn thành **Level 1** (*lập trình thủ tục* với một file lớn) và **Level 2** (*lập trình hướng đối tượng* với cấu trúc nhiều file, interface, GUI), các bạn đã thực hành được hai mẫu hình lập trình tương ứng với hai mức trừu tượng hoá: *trừu tượng hoá thủ tục* (chia chương trình thành các hàm rời) và *trừu tượng hoá dữ liệu* (đóng gói dữ liệu và hành vi trong các lớp).

Level 3 là bước cuối cùng trong lộ trình ba mẫu hình: *lập trình hàm* (*Functional Programming – FP*). Khác với hai level trước, trọng tâm của level này không nằm ở giao diện hay quy mô mã nguồn, mà ở **cách suy nghĩ về tính toán**: chương trình được nhìn như một *chuỗi biến đổi dữ liệu* thay vì một dãy lệnh sửa trạng thái. Đây cũng là cơ sở để các bạn làm quen với khái niệm *trừu tượng hoá điều khiển* (*control abstraction*) – truyền hàm như giá trị, đóng gói chiến lược tính toán bằng lambda và closure, ráp các bước xử lý thành pipeline.

Mục tiêu chính của Level 3 bao gồm:

- **Thực hành lập trình hàm**: hiện thực hoá bốn nguyên lý cốt lõi (*hàm thuần khiết, dữ liệu bất biến, hàm bậc cao, tổ hợp hàm*) trong một dự án thực tế đã có sẵn,
- **Trừu tượng hoá điều khiển**: sử dụng các cơ chế truyền tham số khác nhau ví dụ như (*value, reference, lambda capture*) cùng với hàm bậc cao và closure để diễn đạt các chiến lược tính toán mà không lặp lại logic,
- **Cải tiến thiết kế đã làm ở Level 2**: loại bỏ các phụ thuộc toàn cục (*Logger, generator*), tách rõ *lõi thuần* khỏi *vỏ tác dụng phụ* (*functional core / imperative shell*),
- **Khai thác thế mạnh của FP**: hoặc theo hướng *tính tin cậy cao* (chứng minh tính chất của hàm thuần bằng quy giản β và quy nạp), hoặc theo hướng *hiệu năng song song* (tận dụng tính bất biến để chạy song song an toàn).

Lưu ý quan trọng về phạm vi của Level 3. Đây là level **tuỳ chọn** (*bonus / optional*) – không bắt buộc tất cả các bạn phải làm. Level này phù hợp với các bạn đã hoàn thành tốt Level 2 và muốn thử sức nâng cao hơn. Đề bài chia thành ba mức (**cơ bản – vận dụng – nâng cao**) với độ khó tăng dần và *cộng dồn* – mức cao bao hàm mức thấp – để mỗi bạn có thể chọn dừng lại ở mức phù hợp với năng lực và quỹ thời gian của mình.

Lưu ý về giao diện. Khác với Level 2 (yêu cầu cả Terminal và SDL), Level 3 *không yêu cầu* giao diện đồ hoạ. Phần lớn nội dung tập trung vào các hàm thuần và logic xử lý dữ liệu, do đó giao diện Terminal là đủ để minh hoạ. Các bạn nào đã có sẵn SDL từ Level 2 vẫn được khuyến khích chạy cả hai giao diện – starter code đã giữ tương thích.

II. Mục tiêu học tập (Learning Objectives)

Bài tập này được thiết kế nhằm giúp các bạn củng cố và vận dụng các kiến thức đã học trong phần trừu tượng hoá điều khiển và lập trình hàm. Làm quen với những phiên bản mới nhất của ngôn ngữ C++ trong việc hỗ trợ mẫu hình lập trình hàm. Tách logic xử lý của chương trình riêng biệt và thử sức với thế mạnh của lập trình hàm. Sau khi hoàn thành Level 3, các bạn được kỳ vọng đạt được các mục tiêu sau.

II.1 Hiện thực hoá bốn nguyên lý của lập trình hàm

Trong Level này, các bạn sẽ chuyển một phần đáng kể của codebase từ phong cách *mệnh lệnh (imperative)* sang phong cách *hàm (functional)*. Việc chuyển đổi không chỉ là thay cú pháp `for` bằng `std::transform`, mà phải thể hiện rõ **bốn nguyên lý cốt lõi** đã được trình bày trong Bài 8:

- **Hàm thuần khiết (Pure function):** cùng đầu vào luôn cho cùng đầu ra; không gây tác dụng phụ quan sát được (không log, không IO, không sửa biến toàn cục),
- **Dữ liệu bất biến (Immutability):** thay vì sửa dữ liệu cũ, ta tạo ra giá trị mới; tham số không bao giờ bị mutate sau khi truyền vào,
- **Hàm bậc cao (Higher-order function):** hàm được đối xử như giá trị – truyền vào, trả về, lưu trữ; thay thế cơ chế đa hình kế thừa của OOP bằng truyền hàm trực tiếp,
- **Tổ hợp hàm (Composition / Pipeline):** ráp các hàm nhỏ thành xử lý lớn theo dạng `filter` → `map` → `reduce`, hoặc `pipe(f, g, h)`.

Chương trình của các bạn cần **thể hiện được các nguyên lý này một cách có chủ đích**, kèm minh chứng cụ thể trong báo cáo (đường dẫn file, số dòng, đoạn pipeline tiêu biểu). Tránh việc chỉ liệt kê lý thuyết chung chung mà không đối chiếu với code đã viết.

II.2 Áp dụng nguyên lý trừu tượng hoá điều khiển

Nếu Level 1 thực hành *trừu tượng hoá thủ tục* (chia logic thành hàm) và Level 2 thực hành *trừu tượng hoá dữ liệu* (đóng gói dữ liệu thành lớp), thì Level 3 đưa các bạn đến với *trừu tượng hoá điều khiển (control abstraction)* – một dạng trừu tượng hoá mạnh và linh hoạt hơn.

Nội dung trọng tâm bao gồm:

- **Cơ chế truyền tham số:** hiểu được sự khác biệt giữa các cơ chế như truyền theo giá trị (`by value`), truyền theo tham chiếu (`by reference`) và *lambda capture*

(by value hay by reference). Mỗi cách truyền có hệ quả khác nhau về tính bất biến, vòng đời và hiệu năng,

- **Lambda và closure:** sử dụng hàm ẩn danh để biểu diễn chiến lược tính toán; hiểu rõ *closure* là “*hàm + môi trường được capture*”,
- **Hàm bậc cao:** viết các hàm nhận hàm là tham số hoặc trả về hàm như kết quả (makeBot, measureExecutionTime, pipe, compose),
- **Currying và partial application:** biến hàm nhiều tham số thành chuỗi hàm một tham số – thuận lợi khi muốn “cố định” một phần đối số trong pipeline.

II.3 Cải tiến thiết kế đã làm ở Level 2

Level 3 không chỉ là về FP. Đây cũng là cơ hội để các bạn nhìn lại thiết kế của Level 2 và **khắc phục các điểm chưa tốt**. Một số ví dụ điển hình:

- Logger ở Level 2 là biến inline toàn cục – vi phạm nguyên tắc *Dependency Inversion*. Level 3 yêu cầu **loại bỏ Logger toàn cục** và *inject* qua constructor,
- generator (std::mt19937) cũng là toàn cục – gây ra “*side-effect ngầm*” khiến hàm sử dụng nó không còn pure. Level 3 yêu cầu truyền Rng& làm tham số,
- Engine ở Level 2 *mutate* bàn cờ trực tiếp trong vòng lặp game – không phù hợp với tinh thần dữ liệu bất biến. Level 3 yêu cầu thay bằng cơ chế state = applyTurn(state, move) với mỗi lượt sinh ra một GameState mới.

Việc cải tiến những điểm này không phải là “làm cho có” – nó giúp toàn bộ tăng core/ thực sự pure, mở đường cho các phần phía sau (chứng minh hoặc song song hoá).

II.4 Khai thác thế mạnh của lập trình hàm

Sau khi đã có một codebase với các hàm thuần và dữ liệu bất biến, các bạn sẽ *trải nghiệm* được hai trong số những thế mạnh nổi bật nhất của FP:

- **Tính tin cậy cao:** hàm thuần là *quan hệ toán học* từ input sang output, do đó có thể *chứng minh* các tính chất của nó bằng kỹ thuật chuẩn trong tính toán lambda (quy giản β , equational reasoning, quy nạp cấu trúc) – xem Bài 9 và minh chứng trong slide AXD301,
- **Dễ song song:** pure + immutable nghĩa là không có *shared state* – không có *race condition* – có thể chạy song song một cách an toàn mà không cần mutex.

Hai mục tiêu này tương ứng với hai *nhánh* của mức cao nhất ở phần yêu cầu (xem mục III). Các bạn cần chọn **một trong hai** để khẳng định mình thực sự hiểu vì sao FP được sử dụng trong các hệ thống đòi hỏi tin cậy hoặc hiệu năng cao.

Đối với bài tập này, việc hoàn thiện chức năng vẫn quan trọng (chương trình phải pass grader), nhưng **phần đáng giá nhất nằm ở khả năng tổ chức lại tư duy** – chuyển từ “làm thế nào” (mệnh lệnh) sang “cái gì” (hàm). Các bạn được khuyến khích trải nghiệm chậm rãi và phản tư qua từng phần trong báo cáo.

III. Mã nguồn cung cấp (Starter Code)

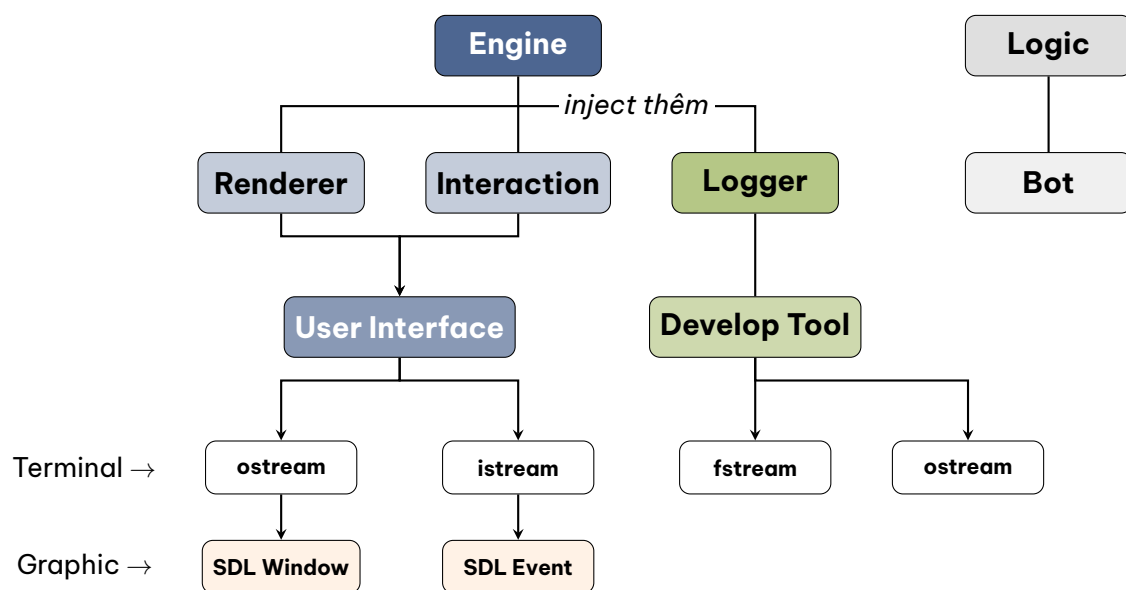
Để giúp các bạn tập trung vào nội dung FP, một thư mục **starter code** đã được cung cấp trong:

level-3/src/

Cấu trúc thư mục được tổ chức lại theo nguyên tắc *functional core / imperative shell*, khác đáng kể so với Level 2.

III.1 Tư duy thiết kế (Design Mindset)

Tinh thần thiết kế của Level 3 có thể tóm gọn trong một câu: **tách rõ phần thuần khỏi phần có tác dụng phụ**. Khi chuyển đổi từ kiến trúc Level 2 sang Level 3, ta được kiến trúc tổng quan như sau:



Hình 1: Kiến trúc chương trình level-3

Có thể thấy rằng, phần bên trái bao gồm những thành phần có tác dụng phụ (side-effect), đây là phần cần lập trình theo phong cách tuần tự (imperative). Phần này sẽ được gọi là *shell*. Ngược lại, bên phải bao gồm các phần thuần túy là logic xử lý dữ liệu. Các thành phần trong này đều không có tác dụng phụ. Phần này được gọi là *core*.

Functional core / Imperative shell. Một codebase FP-friendly thường được chia thành hai vùng:

- **Functional core (lõi thuần):** chứa toàn bộ logic xử lý dữ liệu. Mọi hàm trong vùng này đều *pure*: không IO, không log, không random, không global. Đây là

vùng dễ test, dễ suy luận, và là vùng duy nhất ta có thể *chứng minh* hoặc *song song hoá*,

- **Imperative shell (vỏ tác dụng phụ):** chứa mọi thứ *phải* có tác dụng phụ: IO, render, log, đọc input. Vỏ này gọi xuống lõi để lấy kết quả tính toán, rồi đẩy ra ngoài.

Trong starter code, hai vùng này được hiện thực hoá bằng hai thư mục `src/core/` và `src/shell/`:

```
src/core/          -- PURE: types, logic, bot_pure, pipeline, parallel
src/shell/         -- SIDE-EFFECT: logger, rng, engine
src/interface/     -- abstract interface (level 2)
src/terminal/      -- impl IO terminal (level 2, doi chu ky Board)
src/utils/         -- config, helper
src/main.cpp       -- composition root: Logger/RNG/Renderer
```

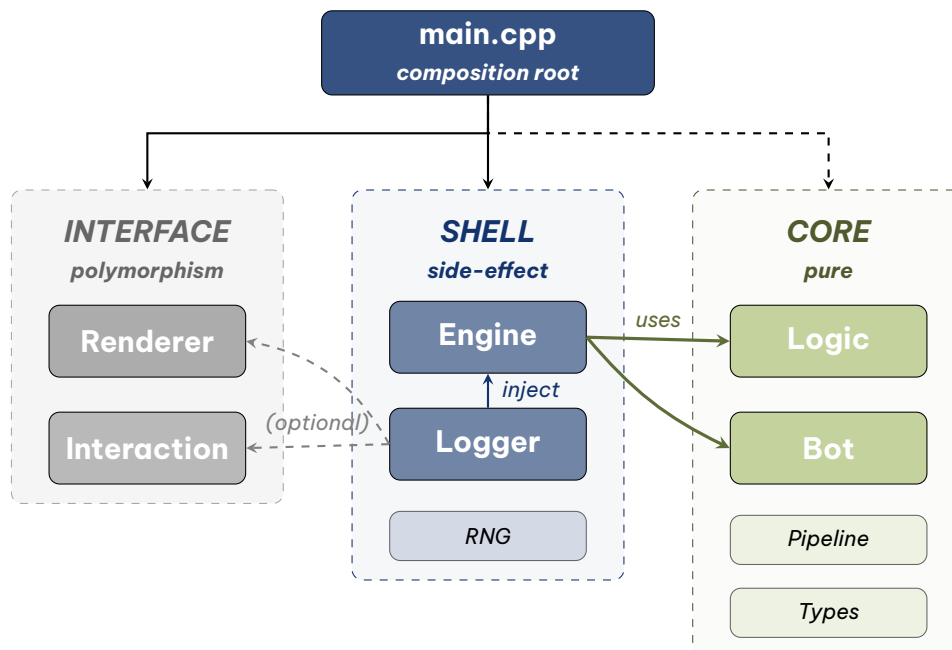
Nguyên tắc gắt gao của tầng core/. Để vùng lõi thực sự thuần, các quy ước sau **phải được tuân thủ**:

- Không `#include <iostream>`, không `cout/cin`,
- Không gọi Logger ở bất kỳ đâu (kể cả qua tham chiếu),
- Không truy cập biến toàn cục (`generator`, `min_level`, ...),
- Mọi hàm nhận tham số `const T&`; nếu cần “thay đổi” thì *trả về* giá trị mới,
- Các hàm cần ngẫu nhiên thì nhận `Rng&` làm tham số – không lấy từ `global`.

Các bạn cần *kiểm soát* tinh thần này khi viết code – nếu không, các phần sau (mức 2 và mức 3) sẽ không thực hiện được.

III.2 Cấu trúc chương trình (Program Structure)

Sơ đồ tổng quát Cấu trúc chương trình về mặt tổng quát có thể biểu diễn như sau:



Hình 2: Cấu trúc level-3 theo mô hình functional core / imperative shell.

Trong đó, hai vùng lớn nét đứt màu **xanh dương** và **xanh lá** biểu diễn ranh giới giữa *imperative shell* và *functional core*. Nhóm **Shell** chứa các thành phần side-effect (Engine, Logger, RNG). Bên cạnh đó, nhóm **Core** chứa các pure logic (Logic, Bot, Pipeline, Types). Vùng **Interface** được tách riêng để gom các thành phần đa hình (Renderer, Interaction).

Mũi tên **xanh đậm** biểu diễn *dependency injection bắt buộc* (Logger được inject vào Engine từ `main.cpp`); mũi tên **nét đứt** biểu diễn inject tùy chọn cho Interface; mũi tên **xanh lá** biểu diễn việc Engine gọi các pure function trong Core.

Một số điểm cần nhấn mạnh:

- `main.cpp` là *composition root* – nơi DUY NHẤT khởi tạo Logger, Rng, Renderer, Interaction và *inject* xuống Engine. Không có biến toàn cục nào nằm ngoài hàm `main()`,
- Hai khối *lớn* ở mỗi vùng là tâm điểm bài tập: Engine & Logger ở SHELL, Logic & Bot ở CORE – tương ứng với 3 task của Mức 1 (xem mục IV.1) và 2 đối tượng của Mức 2 (xem mục IV.2),
- Khối *lớn* Interface là phần đã được hoàn thiện trong bài tập lớn Level 2. Có thể inject Logger vào để sử dụng.
- Hai khối *nhỏ* (Pipeline, Types ở CORE; RNG ở SHELL) là phần phụ trợ – đã được cung cấp sẵn hoặc chỉ cần ánh xạ lại từ Level 2.

III.3 Thành phần mã nguồn

Bảng dưới đây tóm tắt vai trò từng thư mục/file. Cột “Mức yêu cầu” chỉ ra mức nào mà bạn cần can thiệp vào file đó.

Bảng 1: Thành phần mã nguồn level-3 và mức yêu cầu

File / Thư mục	Vai trò	Mức
CORE		
src/core/types.h	Định nghĩa Board, Move, GameState (immutable)	—
src/core/pipeline.h	Cung cấp pipe, compose, curry (header-only)	—
src/core/logic.{h,cpp}	Pure functions: initBoard, applyMove, checkWin, ...	1
src/core/bot_pure.{h,cpp}	BotFn = (state, rng) → Move + factory	1 – 2
src/core/parallel.{h,cpp}	evaluateAllMoves_seq/par cho bot Hard	3 (B)
src/core/recursion_combinator.cpp	Demo Y combinator (file độc lập)	3 (A)
SHELL		
src/shell/logger.{h,cpp}	Class Logger (instance-based, không global)	1
src/shell/rng.h	Alias Rng = std::mt19937 + makeRng(seed)	1
src/shell/engine.{h,cpp}	Game loop (mức 2: không mutate board)	1 – 2
INTERFACE		
src/interface/	Abstract I_Renderer, I_Interaction	—
src/terminal/	Impl Terminal (chép từ level 2)	1
UTILS / ROOT		
src/utils/config.{h,cpp}	Parse argv -> RunConfig	—
src/utils/helper.{h, tpp}	measureExecutionTime nhận LogSink (HOF)	—
src/main.cpp	Composition root: tạo & inject mọi dependency	—

Pipeline có sẵn – src/core/pipeline.h. File này được cung cấp sẵn (các bạn không phải tự cài), bao gồm các combinator chuẩn:

```
1 namespace fp {
2     // pipe(f1, f2, f3) = lambda x => f3(f2(f1(x)))
3     template <typename F, typename G, typename... Rest>
4     auto pipe(F&& f, G&& g, Rest&&... rest);
5
6     // compose(f, g) = lambda x => f(g(x))    (toan hoc: f o g)
7     template <typename F, typename G, typename... Rest>
8     auto compose(F&& f, G&& g, Rest&&... rest);
9
10    // curry(f)(x)(y) = f(x, y)
11    template <typename F>
12    auto curry(F f);
13
14    // identity, constant
15 }
```

Các bạn *nên đọc hiểu* kĩ các combinator này trước khi tiến hành làm mức 2 – chúng làm pipeline trở nên gọn và rõ hơn.

Kiểu dữ liệu GameState – src/core/types.h. Trong Level 2, trạng thái game được lưu rải rác (board ở GameSetup, lượt hiện tại ở biến cục bộ trong playGame()), kết quả ở GameResult). Level 3 gom toàn bộ trạng thái vào một struct duy nhất:

```
1 struct GameState {
2     Board board;           // ban co (immutable)
3     int currentPlayer;    // 0 hoặc 1
4     int turn;             // số lượt đã đi
5     int winner;           // -1 nếu chưa kết thúc / hoặc; 0 hoặc 1 nếu có
                           // người thắng
6     bool isFinished;      // game đã kết thúc hay chưa
7 };
```

GameState đóng vai trò *input & output* của hàm applyTurn(state, move, goal) ở Mức 2 – mỗi lượt sinh ra một GameState mới (bất biến), *không* sửa state cũ. Đây là viên gạch nền để Engine ở Mức 2 diễn đạt game loop dạng fold/đệ quy đuôi:

```
1 GameResult loop(GameState s) {
2     if (s.isFinished) return resultOf(s);
3     Move m = nextMove(s);
4     return loop(applyTurn(s, m, goal)); // tail call
5 }
```

Có thể coi GameState là *snapshot* của game tại một thời điểm: mỗi turn ta chụp một bức ảnh mới, các bức ảnh cũ vẫn còn nguyên (mở ra khả năng undo, replay, time-travel debugging mà Level 2 không có).

Mapping từ Level 2 sang Level 3. Bảng dưới đây chỉ ra thay đổi cụ thể của 3 nhóm đối tượng quan trọng nhất, theo từng mức (xem chi tiết yêu cầu từng mức tại Mục IV.1, IV.2, IV.3).

Bảng 2: Mapping 3 nhóm đối tượng chính qua Level 2 và 3 mức của Level 3.

Mức	<i>Logic::*</i> (quy mô nhỏ)	<i>Engine, Logger</i> (quy mô lớn)	<i>Bot</i> (thuật toán)
Level 2	void makeMove(char board[][N], r, c, s) – mutate tham số	Logger::log(...) global; Engine mutate board trong loop	class Bot virtual + 3 lớp con; dùng generator global
Mức 1 (cơ bản)	Board applyMove(const Board&, Move, char) – pure, trả về board mới	Logger <i>inject</i> qua constructor; Engine vẫn mutate	Bot LV1 cài bằng pipeline HOF, RNG inject
Mức 2 (vận dụng)	Ổn định – được Engine gọi như pure transformation	Engine <i>không</i> mutate: state = applyTurn(state, move); loop là fold/đệ quy đuôi	Bot bất kỳ (bạn tự chọn): áp dụng pipe/compose/curry
Mức 3 (nâng cao)	Đối tượng để <i>chứng minh tính chất</i> (vd: số ô tăng đúng 1) ở nhánh A (IV.3)	(giữ nguyên ở mức 2 – không đối tượng cần cải tiến thêm)	Bot Hard song song hoá: evaluateAllMoves_seq vs _par, benchmark, ở nhánh B (IV.3)

IV. Hướng dẫn triển khai (Implementation Guide)

Phần này mô tả **ba mức yêu cầu** của Level 3, theo thứ tự độ khó tăng dần. Các mức được thiết kế *cộng dồn*: mức cao bao hàm mức thấp. Các bạn có thể chọn dừng lại ở mức phù hợp với năng lực và quỹ thời gian.

Mức 1 (cơ bản) → Mức 2 (vận dụng) → Mức 3 (nâng cao)

IV.1 Mức 1 – Cơ bản (FP cú pháp)

Mức này là **điều kiện cần** để bài làm được tính là Level 3. Các bạn không cần thay đổi kiến trúc tổng thể của chương trình – chỉ cần thể hiện **các nguyên lý cú pháp của FP** qua ba task cụ thể, mạch lạc.

Mức độ: Mức 1 phù hợp với *phần lớn sinh viên*, kể cả các bạn chỉ làm được Bot Easy ở Level 2.

Task 1 – FP-hoá Logic :: *. Namespace Logic được đổi tên thành core. Yêu cầu chuyển toàn bộ các hàm có trong file `src/core/logic.{h,cpp}` từ phong cách mệnh lệnh sang phong cách hàm thuần:

- Mọi hàm phải PURE: nhận `const Board&`, không gọi Logger, không truy cập biến toàn cục,
- Hàm `applyMove` **trả về Board mới**, không sửa tham số. Đây là minh chứng cho tính *bất biến*,
- Cài đặt ít nhất *một* hàm bằng **pipeline** `filter → map → reduce` (gợi ý: cài đặt hàm `enumerateValidMoves` dùng `std::ranges::views::filter`),
- Cài đặt ít nhất *một* hàm bằng **đệ quy** (gợi ý: `countSymbol` với đệ quy đuôi). Báo cáo cần ghi chú compiler có thực hiện TCO không.

Task 2 – Higher-order function cho Bot LV1 (Easy). Không sử dụng trừu tượng hoá dữ liệu cho Bot, mà áp dụng hàm bậc cao. Trong `src/core/bot_pure.{h,cpp}`, cài đặt:

- Hàm `easy(state, rng)` chọn ngẫu nhiên một ô trống. Yêu cầu cài đặt bằng *pipeline* (`enumerateCells → filter empty → pick random`). Tuyệt đối không dùng vòng `for` tích lũy kiểu mệnh lệnh,
- Hàm `makeBot(level, goal)` là *factory trả về một hàm* – thể hiện rõ HOF: hàm trả về hàm. Mỗi nhánh case trả về một `lambda` đóng gói `goal` vào *closure*,

- Tham số Rng& bắt buộc – KHÔNG dùng generator toàn cục.

Bot Medium và Hard có thể tạm để NotImplementedException nếu các bạn chưa làm chúng ở Level 2 – mức 1 chỉ yêu cầu Bot Easy.

Task 3 – Loại bỏ Logger toàn cục. Trong src/shell/logger.{h,cpp}, Logger đã được thiết kế là một *class* (không phải biến inline global như Level 2). Việc còn lại là cập nhật toàn bộ chỗ gọi:

- Trong Engine, logger_ được nhận từ constructor và lưu lại làm con trở thành viên. Mọi Logger::log(...) ở Level 2 phải đổi thành logger_→log(...),
- Trong các hàm helper (ví dụ measureExecutionTime), thay vì gọi lớp Logger trực tiếp. Thay vào đó, hàm này nhận một tham số khác với tên là LogSink = std::function<void(const std::string&)> để truyền một hàm bậc cao hơn bao lấy hàm logger→log() vào – đây cũng là một ví dụ HOF,
- main.cpp (đã có sẵn) tạo Logger cục bộ rồi inject xuống Engine – không có biến inline ở scope toàn cục.

Yêu cầu báo cáo cho Mức 1. Báo cáo cần chỉ ra (kèm số dòng / tên file):

- Đây là **pure function** (ít nhất 3 ví dụ),
- Đây là **higher-order function** (ít nhất 2 ví dụ),
- Đây là **đệ quy** mà bạn đã viết (kèm thảo luận về TCO),
- Logger và RNG được **inject** ở những vị trí nào (chỉ ra constructor/factory).

IV.2 Mức 2 – Vận dụng (FP có chủ đích)

Mức này dành cho các bạn đã hoàn thành Mức 1 và muốn thực sự *vận dụng* FP để cải tiến thiết kế ở quy mô lớn. Mức 2 tập trung vào **hai đối tượng cụ thể**: Engine (quy mô lớn) và Bot (thuật toán).

Mức độ: Mức 2 phù hợp với sinh viên *khá – giỏi*, đặc biệt các bạn đã hoàn thiện Bot Medium hoặc Hard ở Level 2.

Đối tượng 1 – Engine: Functional Core / Imperative Shell. Mục tiêu: Engine hoàn toàn không mutate Board nữa.

- Thay vì Logic::makeMove(board, r, c, s) (mutate), dùng:

```
1 gameState_ = core::applyTurn(gameState_, move, goal);
```

Mỗi lượt sinh ra một GameState mới; trạng thái cũ *vẫn còn nguyên* (có thể dùng cho undo, replay, ...),

- Vòng lặp playGame() được khuyến khích viết theo phong cách *đệ quy đuôi* hoặc *fold*:

```

1 GameResult loop(GameState s) {
2     if (s.isFinished) return resultOf(s);
3     Move m = nextMove(s);
4     return loop(core::applyTurn(s, m, goal)); // tail call
5 }

```

Đây là minh họa rõ ràng cho ý “*game loop là một fold qua chuỗi moves*”,

- Tách rõ **phần pure** (nextMove, applyTurn, resultOf) khỏi **phần shell** (render, input, log). Báo cáo cần chỉ ra ranh giới này – ở dòng nào trong engine.cpp.

Đối tượng 2 – Bot: composition, closure, currying. Các bạn *tự chọn* áp dụng FP vào bot nào (Medium hoặc Hard, hoặc cả hai) – tùy vào thuật toán đã làm được ở Level 2. Yêu cầu thể hiện được **ít nhất hai** trong các kỹ thuật sau:

- **Composition (tổ hợp hàm):** dùng `fp::pipe(...)` hoặc `fp::compose(...)` để ráp các bước nhỏ thành một bot. Ví dụ Bot Medium:

```

1 auto choose = fp::pipe(
2     enumerateValidMoves, // Board -> vector
3     <Move>
4     filterFirst(isWinningMove(state, goal)), // -> optional<
5     <Move>
6     orElse(filterFirst(isBlockingMove(...))),
7     orElse(easy(state, rng))
8 );
9 Move m = choose(state.board);

```

- **Closure:** đóng gói state, goal, symbol vào lambda để truyền cho hàm bậc cao – khác với truyền tham số tường minh ở mỗi lần gọi,
- **Currying / partial application:** dùng `fp::curry(...)` để cố định một số tham số của hàm đánh giá:

```

1 auto evalForX = fp::curry(evaluate)(SYMBOL_X);
2 // (Board, Symbol) -> int
3 //      => Board -> int

```

Yêu cầu báo cáo cho Mức 2.

- Trình bày sơ đồ *functional core / imperative shell* của Engine,
- Chỉ ra trong engine.cpp đâu là phần pure, đâu là phần shell – bằng số dòng cụ thể,
- Chỉ ra trong bot mình đã chọn: nơi áp dụng *composition, closure, currying*; giải thích vì sao những kỹ thuật này làm code rõ hơn so với cách viết bằng OOP polymorphism ở Level 2.

IV.3 Mức 3 – Nâng cao (FP thực sự mạnh)

Mức cao nhất. Các bạn chọn **đúng một trong hai nhánh** A hoặc B – không yêu cầu cả hai. Hai nhánh có trọng số tương đương trong rubric.

Mức độ: Mức 3 phù hợp với sinh viên *xuất sắc*, đã hoàn thiện Bot Hard ở Level 2 và đã vượt qua Mức 2 ở Level 3.

Nhánh A – Chứng minh toán học trên mã nguồn. Vận dụng kiến thức từ Bài 9 (Phép tính lambda, quy giản β , equational reasoning, quy nạp cấu trúc) để *chứng minh* các tính chất của các pure function đã viết ở Mức 2.

Yêu cầu cụ thể:

- Chọn **ít nhất 2 pure function** trong codebase. Gợi ý: `applyMove`, `countSymbol`, một bot heuristic đơn giản (như `easy`, `medium`) hoặc một combinator bất kỳ bên trong `pipeline.h`. Việc lựa chọn hàm để chứng minh càng khó, điểm đánh giá sẽ càng cao.
- Cho mỗi hàm:
 1. **Đặc tả toán học:** viết signature dạng pseudocode kiểu ML hoặc ký pháp λ , nêu *invariant* hoặc tính chất cần chứng minh,
 2. **Chứng minh tính chất.** Có thể chọn một trong các kỹ thuật:
 - Quy giản β *step-by-step* cho hàm đơn giản dạng λ ,
 - Quy nạp cấu trúc trên list hoặc độ sâu đệ quy. Ví dụ: chứng minh

$$\text{countSymbol}(\text{applyMove}(b, m, s), s) = \text{countSymbol}(b, s) + 1$$

với điều kiện `isValidMove(b, m) = true`,

- *Equational reasoning*: chứng minh hai cài đặt khác nhau cùng giá trị, ví dụ

$$\text{map } f \circ \text{map } g = \text{map } (f \circ g).$$

3. Chứng minh phải được trình bày cẩn thận trong báo cáo (LaTeX), kèm tham chiếu rõ ràng tới đoạn code (file & số dòng).
- **Property-based test:** mỗi tính chất được chứng minh phải đi kèm một test thực tế trên ≥ 100 input ngẫu nhiên. Đây là cách kiểm chứng rằng *định lý* của bạn không bị code phản bội,
 - **(Khuyến khích, không bắt buộc)** các bạn có thể mở rộng nghiên cứu thêm phần `src/core/recursion_combinator.cpp` để minh họa Tổ hợp Y – “*đệ quy không cần tên*”. File này chỉ là demo độc lập, không cần link vào game.

Nhánh B – Song song hoá Bot Hard. Vận dụng đặc tính “*pure* $\rightarrow$ *no shared state* $\rightarrow$ *không cần khoá*” để chạy Bot Hard song song và đo đạc speedup.

Yêu cầu cụ thể (cài đặt trong `src/core/parallel.{h,cpp}`):

- Hoàn thiện hàm `evaluateMove(state, move, symbol, goal)`, tuân thủ tính chất *pure, deterministic*. Có thể là minimax 1 tầng hoặc heuristic mạnh,
- Hoàn thiện hai phiên bản:
 - Hàm `evaluateAllMoves_seq(state, sym, goal)` là baseline, chạy tuần tự,
 - Hàm `evaluateAllMoves_par(state, sym, goal, num_threads)` chạy song song. Có thể chọn:
 - ★ `std::execution::par` với `std::transform`,
 - ★ `std::async + std::future` (mỗi nước một task),
 - ★ Tự xây thread pool đơn giản.
- **Chứng minh tính đúng đắn:** test rằng seq và par cho *cùng output* với cùng input, trên ít nhất 100 board ngẫu nhiên. Đây là minh chứng “*pure thì song song an toàn*”,
- **Benchmark:** chạy mỗi phiên bản ≥ 30 lần trên 3 cấu hình board: 5×5 , 8×8 , 12×12 . Ghi lại thời gian (median + IQR) qua log,
- **Báo cáo:** vẽ biểu đồ thời gian, bảng số liệu, và *giải thích*:
 - Vì sao việc song song hoá lại đúng đắn (do pure + immutable),
 - Speedup đạt được trên máy thử nghiệm là bao nhiêu,
 - Vì sao đôi khi song song hoá *không* cho speedup (overhead, kích thước task quá nhỏ, false sharing, ...).
- **(Khuyến khích, không bắt buộc)** thử nghiệm thêm chiến lược *lazy evaluation* bằng `std::function` trả về closure – minh hoạ ý “call-by-name” trong slide nâng cao.

Yêu cầu báo cáo cho Mức 3.

- **Nhánh A:** ít nhất 1 chứng minh đầy đủ (đẹp, có các bước rõ ràng), kèm property-based test minh chứng,
- **Nhánh B:** biểu đồ benchmark, bảng số liệu thô, kết luận về speedup.

V. Hướng dẫn thực nghiệm (Experiment Instructions)

V.1 Cài đặt môi trường

Yêu cầu công cụ.

- Trình biên dịch hỗ trợ **C++20** ($g++ \geq 10$, $clang++ \geq 12$, MSVC 2019+),
- `cmake` ≥ 3.16 ,
- `python3` (chạy grader),
- (Tuỳ chọn) `SDL2` + `SDL2_ttf` nếu giữ phần GUI từ Level 2,
- (Tuỳ chọn, mức 3 nhánh B) thư viện `libtbb` nếu dùng `std::execution::par` trên Linux.

Cài thư viện song song trên Linux (cho mức 3 nhánh B). Sử dụng câu lệnh sau:

```
1 sudo apt install libtbb-dev    # Ubuntu/Debian
```

Sau đó bỏ comment 2 dòng `find_package(TBB ...)` trong `CMakeLists.txt`.

V.2 Biên dịch chương trình

Cách 1 – CMake (khuyến dùng). Tương tự Level-2, nhưng có thêm tùy chọn bỏ qua SDL như sau:

```
1 cd level-3
2 mkdir build && cd build
3 cmake -DUSE_SDL=OFF ..      # bỏ qua SDL
4 cmake --build .
```

Bật SDL nếu muốn giữ GUI từ Level 2:

```
1 cmake -DUSE_SDL=ON ..
```

Cách 2 – gọi g++ trực tiếp. Nếu bạn không muốn sử dụng `cmake`, có thể sử dụng trực tiếp `g++` như sau:

```
1 cd level-3
2 g++ -std=c++20 -Wall -Wextra -Isrc -pthread \
3     src/main.cpp src/utils/config.cpp \
4     src/shell/logger.cpp src/shell/engine.cpp \
5     src/interface/i_renderer.cpp src/interface/i_interaction.cpp \
6     src/terminal/renderer.cpp src/terminal/interaction.cpp \
7     src/core/logic.cpp src/core/bot_pure.cpp src/core/parallel.cpp \
8     -o game
```

Build demo Y combinator (mức 3 nhánh A, tham khảo). Biên dịch demo chương trình Y combinator, sử dụng lệnh sau:

```
1 cd build
2 cmake --build . --target y_demo
3 ./y_demo
```

V.3 Chạy chương trình

```
1 ./game --help           # in help
2 ./game                 # interactive (terminal)
3 ./game -j -i input.txt  # judge mode (output cho grader)
4 ./game -p              # bat parallel bot HARD (muc 3 nhanh B)
5 ./game -s 12345        # dat seed RNG
6 ./game -v              # log chi tiet DEBUG
```

V.4 Chạy mã chấm điểm (Grader)

Level 3 **tái sử dụng** grader.py và bộ testcase/ của Level 1/2 – vì logic cơ bản không đổi, chỉ thay đổi cách cài đặt. Các bạn cần đảm bảo chương trình của mình **vẫn pass toàn bộ testcase** sau khi refactor.

```
1 cd level-3
2 python grader.py --target build/game
```

V.5 Đo đạc cho Mức 3 nhánh B

Đối với các bạn chọn nhánh B, các bước thực nghiệm gợi ý:

1. Viết một script bash hoặc Python lặp ≥ 30 lần lần lượt với `--parallel` và không có `--parallel`, ghi lại thời gian từ log,
2. Chạy với 3 cấu hình board: 5×5 , 8×8 , 12×12 ,
3. Tính median + IQR (interquartile range), vẽ box-plot hoặc bar-chart,
4. Đặt kết quả vào thư mục `logs/` và `results/` (xem mục Nộp bài).

V.6 Ghi nhận kết quả vào báo cáo

Ở Level 3, trọng tâm mình chứng dịch chuyển sang **tính đúng đắn của refactor, ranh giới core/shell** và (nếu có) **kết quả định lượng của song song hoá**. Các bạn cần tổng hợp các nội dung sau vào hai thư mục:

`./results/` – minh chứng cho báo cáo (ảnh, biểu đồ, bảng)
`./logs/` – file log thô của các lần chạy

1. **Ảnh chụp grader pass.** Đây là minh chứng *tối thiểu* cho thấy refactor sang FP không phá vỡ logic cũ:
 - Chạy `python3 grader.py` ở thư mục `level-3/` sau khi build xong,
 - Chụp toàn bộ output (PASS/FAIL từng testcase),
 - Lưu thành `results/grader_pass.png`.
2. **Ảnh chụp giao diện chạy thật.** Tương tự Level 2, nhưng Level 3 không bắt buộc cả hai giao diện – chỉ cần Terminal là đủ:
 - Chụp 3–4 màn hình tiêu biểu: menu chọn cấu hình, board đang chơi, kết quả cuối,
 - Gợi ý cách đặt tên file ảnh: `terminal_menu.png`, `terminal_board.png`, `terminal_result.png`, ...,
 - Nếu các bạn vẫn giữ SDL từ Level 2 và chạy được, ảnh SDL là điểm cộng (đặt theo cặp như Level 2).
3. **File Log – minh chứng Logger đã được inject (Mức 1).** Đây là điểm rất quan trọng của Level 3: Logger không còn là biến toàn cục. File log cần thể hiện:
 - Dòng đầu tiên nội dung `"Logger initialized!"` được xuất hiện ngay sau khi `main.cpp` tạo đối tượng Logger,
 - Các dòng log đo thời gian từ `measureExecutionTime` với label rõ ràng (ví dụ như: `execution time of [bot→getMove()] = 0.00012s`) xuất hiện chỉ khi flag `-v` được bật,
 - Các file log được lưu vào `logs/log_default.txt` (chạy không có `-v`) và `logs/log_verbose.txt` (chạy với `-v`).
4. **(Mức 1 + 2) Trích đoạn code minh họa.** Một vài đoạn code 5–10 dòng đặt *trực tiếp trong báo cáo* (bằng `lstlisting` hoặc ảnh chụp màn hình IDE), kèm chú thích đường dẫn file + số dòng:
 - Một đoạn code pure function,
 - Một đoạn code sử dụng hàm bậc cao, lambda,
 - Một pipeline filter → map → reduce của bạn,
 - Một ví dụ closure / currying ở Bot,
 - Đoạn fold/đệ quy đuôi ở `Engine::playGame`.
5. **(Mức 3 nhánh A) Tài liệu chứng minh.**
 - Trình bày chứng minh trực tiếp trong báo cáo (LaTeX hoặc ảnh chụp viết tay), – đây là phần rất quan trọng của báo cáo,
 - Đính kèm output của property-based test (`logs/property_test.txt`) – minh chứng rằng tính chất bạn chứng minh vẫn đúng trên 100+ random input.

6. (Mức 3 nhánh B) Benchmark song song hoá.

- File log thô sau khi benchmark của mỗi cấu hình: logs/log_seq_5x5.txt, logs/log_par_5x5.txt, logs/log_seq_8x8.txt, ... (≥ 30 lần đo mỗi file),
- Bảng tổng hợp dạng CSV: results/benchmark_table.csv với các cột board_size, mode, median_ms, q1_ms, q3_ms,
- Biểu đồ trực quan: results/benchmark_chart.png (box-plot hoặc bar-chart so sánh seq vs par).

Nguyên tắc chung: **mọi minh chứng phải có thể tái lập được**. Người chấm có thể chạy lại lệnh tương ứng trên máy của các bạn (qua README) để kiểm tra. Không chấp nhận ảnh đẹp nhưng không có log thô đi kèm.

Lưu ý: không bịa số liệu benchmark. Sự khác biệt giữa seq và par phải khớp với log thô được nộp; người chấm sẽ kiểm tra ngẫu nhiên một vài đoạn log để đối chiếu với bảng/biểu đồ.

VI. Yêu cầu báo cáo (Report Requirements)

Báo cáo của Level 3 đặc biệt quan trọng – vì nội dung FP nặng về tư duy, phần lớn giá trị bài làm nằm ở khả năng *phản tư* và *liên hệ giữa lý thuyết với code thực tế*.

VI.1 Định dạng và Quy định chung

- **Hình thức:** Nộp file định dạng **PDF**. Giữ nguyên cấu trúc nội dung.
- **Công cụ:** Khuyến khích sử dụng L^AT_EX (Overleaf) hoặc MS Word/Google Docs nhưng phải xuất PDF.
- **Yêu cầu:** Nội dung báo cáo phải phản ánh chính xác mã nguồn đã nộp. Mọi hành vi sao chép sẽ bị xử lý theo quy định.
- **Ngôn ngữ:** Tiếng Việt, trích dẫn thuật ngữ Tiếng Anh (lần đầu nhắc đến).
- **Đặt tên:** lưu file báo cáo với định dạng tên sau:

`report_level-3_<HoVaTen>_<mssv>.pdf`

Ví dụ: `report_level-3_NguyenVanA_2502XXXX.pdf`

VI.2 Cấu trúc nội dung (Outline)

Báo cáo cần tuân thủ đúng 06 mục sau:

1. Tổng quan (Overview)

- Thông tin cá nhân (Họ tên, MSSV),
- **Mức đạt được:** Mức 1 / Mức 2 / Mức 3 (nhánh A hoặc B). Đây là thông tin quan trọng – giúp giảng viên chấm đúng phạm vi,
- Tóm tắt ngắn gọn (5 dòng) về cách bạn tổ chức code: tầng core/ chứa gì, tầng shell/ chứa gì, có cải tiến gì so với Level 2.

2. Chi tiết cài đặt (Implementation Details)

Phần này tập trung trả lời các vấn đề kỹ thuật cốt lõi của Level 3. Các bạn không cần chép toàn bộ mã nguồn, chỉ cần đường dẫn file + số dòng + đoạn code tiêu biểu (5–10+ dòng) cho mỗi nội dung. Cụ thể, nội dung báo cáo tập trung vào các đầu mục sau:

- **Tư duy thiết kế core/shell:** Trình bày cách bạn vạch ranh giới giữa core/ (pure) và shell/ (side-effect). Quy ước nào được các bạn áp dụng triệt để (vd: cấm `#include <iostream>` trong core/)? Có vi phạm nào còn tồn đọng không? Vì sao chấp nhận / không chấp nhận?
- **Trình bày từng mức đã đạt:** Bạn đạt mức nào (1 / 2 / 3A / 3B)? Với mỗi mức, mô tả ngắn gọn các thay đổi cụ thể trong code so với Level 2. Có thể dùng bảng đối chiếu “Level 2 – Level 3” của riêng bạn.
- **Cải tiến thiết kế từ Level 2:** Liệt kê các *lỗi thiết kế* ở Level 2 mà bạn nhận ra và đã giải quyết được trong Level 3 (Logger global, generator global, Engine mutate board, coupling chặt, ...). Mỗi lỗi cần kèm: cách bạn sửa, lý do sửa, tác động lên tính testable / mở rộng.
- **Trả lời câu hỏi:** Trả lời các câu hỏi yêu cầu được liệt kê bên dưới. Mỗi câu nên có minh chứng cụ thể (đường dẫn file, số dòng, ảnh chụp đoạn code).
- **(Nếu chọn Mức 3)** Trình bày phần nâng cao:
 - **Nhánh A:** chứng minh tính chất của pure function bạn đã chọn – đặc tả, các bước (quy giản β , quy nạp, equational reasoning), và kết luận. Đính kèm property-based test.
 - **Nhánh B:** kết quả benchmark song song hoá Bot Hard – bảng số liệu, biểu đồ, phân tích speedup theo cấu hình board.

Câu hỏi yêu cầu:

1. **Hàm thuần và bất biến (Mức 1):** chỉ rõ trong code đâu là *pure function* (kèm số dòng), đâu là *side-effect bị cô lập*. Bạn cô lập *side-effect* ở những chỗ nào? Cách kiểm soát ranh giới giữa core/ và shell/ của bạn ra sao?
2. **Hàm bậc cao (Mức 1):** liệt kê ít nhất 3 ví dụ hàm bậc cao trong code của bạn. Ở mỗi ví dụ, mô tả vì sao việc dùng HOF làm code rõ hơn so với cách viết tương đương ở Level 2 (ví dụ: thay *polymorphism kế thừa* bằng *truyền hàm*).
3. **Trừu tượng hoá điều khiển (Mức 1 – 2):** bạn dùng cơ chế nào trong các cơ chế sau: *lambda*, *closure*, *currying*, `std::function`? Tại sao chọn cơ chế đó cho từng tình huống? Có trường hợp nào bạn nhận thấy rõ một cơ chế phù hợp hơn cơ chế còn lại?
4. **Cải tiến thiết kế từ Level 2 (Mức 1 – 2):** bạn đã loại bỏ Logger / RNG global ra sao? Có *lỗi thiết kế* nào tồn tại ở Level 2 mà bạn giải quyết được trong Level 3 không (ví dụ: Engine mutate board, coupling chặt giữa Bot và Logger, không có DI, ...)? Bạn giải quyết bằng cách nào? Việc cải tiến này có tác động gì đến tính testable / mở rộng của code?
5. **(Mức 3 – nhánh A)** Trình bày **ít nhất một chứng minh đầy đủ** cho một pure function bạn đã chọn. Chứng minh cần có: đặc tả, các bước (quy giản β , quy

nạp, hoặc equational reasoning), và kết luận. Đính kèm property-based test thực tế kiểm chứng tính chất.

6. **(Mức 3 – nhánh B)** Trình bày kết quả benchmark song song hoá Bot Hard:

- Bảng số liệu thô (median, IQR) cho 3 cấu hình board,
- Biểu đồ trực quan,
- Kết luận: speedup đạt bao nhiêu? Có cấu hình nào không có speedup? Vì sao?

7. **So sánh ba mẫu hình:** qua hành trình Level 1 → 2 → 3, bạn nhìn nhận *ưu / nhược* của FP so với OOP và procedural ra sao? Khi nào thì FP thực sự phù hợp? Khi nào thì không?

3. Kết quả thực nghiệm (Experiment Results)

Đọc kĩ phần V.6 để nắm được rõ cụ thể nội dung kết quả cần báo cáo. Sau đây là một số lưu ý:

- **Ảnh chụp grader pass** xác nhận chương trình vẫn đúng logic sau khi refactor,
- **Ảnh chụp giao diện chạy thật** không yêu cầu cả hai giao diện, nếu có thay đổi thêm từ level 2 thì nên đưa vào báo cáo,
- **Trích dẫn log** tiêu biểu, đặc biệt là log đo thời gian từ `measureExecutionTime`; phải thể hiện rõ Logger đã được inject (không global);
- **(Mức 3 nhánh B)** Bảng và biểu đồ kết quả benchmark.

4. Nhận xét và tổng kết bài làm

Trả lời ngắn gọn các câu hỏi:

- **Về phong cách lập trình:** sau khi đã làm cả 3 mẫu hình, bạn thấy phong cách FP có điểm mạnh / yếu gì so với OOP và procedural? Trong dự án *tic-tac-toe* cụ thể này, FP có thực sự “thắng” OOP không? Phần nào của codebase phù hợp với FP, phần nào vẫn nên dùng OOP?
- **Về tư duy bất biến:** việc viết code không mutate dữ liệu có làm bạn thấy “khó chịu” lúc đầu không? Khi quen rồi, có lợi ích gì rõ ràng (ví dụ: dễ debug, dễ undo, dễ test)?
- **Về thế mạnh của FP:** qua phần Mức 3 (nếu có làm), bạn cảm nhận thế nào về việc FP có thể *chứng minh* hoặc *song song hoá*? Liệu các thế mạnh này có đủ để biện minh cho những bất tiện của FP (cú pháp, hiệu năng copy)?

- **Về điểm yếu của FP:** thông qua quá trình hoàn thiện bài tập lớn này, bạn có những góc nhìn nào về điểm yếu của phong cách lập trình này? Liệu các điểm yếu này có thể được cải thiện nhờ vào đâu? Và trong tương lai, mẫu hình này có còn được áp dụng nhiều trong các giải pháp lập trình không?
- **Về phong cách lập trình:** Sau bài tập lớn này, các bạn đã được trải nghiệm 3 phong cách lập trình khác nhau. Bạn có phân biệt được những dự án công nghệ / bài toán nào phù hợp với từng kiểu lập trình không? Hãy đưa ra diễn giải và ví dụ của riêng bạn. Trong bối cảnh AI ngày càng hỗ trợ mạnh việc sinh mã nguồn, theo bạn vai trò của lập trình viên trong việc lựa chọn phong cách lập trình và tổ chức lời giải sẽ thay đổi như thế nào?
- **Về thiết kế chương trình:** Trong quá trình làm bài, các bạn cũng đã được tiếp cận nhẹ nhàng với việc thiết kế chương trình trước khi lập trình. Theo bạn, việc thiết kế chương trình đóng vai trò như thế nào trong quá trình phát triển phần mềm? Khi AI có thể tự sinh ra một phần lớn mã nguồn, đâu vẫn là những phần việc mà con người cần trực tiếp đảm nhiệm trong quá trình thiết kế hệ thống?
- **Bài học rút ra:** khó khăn lớn nhất bạn đã vượt qua là gì? Đó là vấn đề thiết kế, vấn đề tư duy FP, hay vấn đề kỹ thuật C++?

5. Ghi chú sử dụng AI (AI Disclosure)

Sinh viên cần trung thực khai báo việc sử dụng các công cụ AI (như ChatGPT, GitHub Copilot,...) trong quá trình làm bài:

- Mục đích sử dụng (ví dụ: hỏi về cú pháp `std::ranges`, gợi ý cấu trúc pipeline, sửa lỗi LaTeX),
- Mức độ đóng góp của AI vào sản phẩm cuối cùng.

Đọc kỹ phần **Chính sách sử dụng AI**, để nắm được các lưu ý quan trọng trước khi tiến hành sử dụng AI trong quá trình làm bài.

6. Nguồn tham khảo (References)

Liệt kê các tài liệu, trang web hoặc thư viện (slide bài 8 / 9, sách, blog, video, ...) đã sử dụng trong quá trình tìm hiểu và thực hiện bài tập.

Báo cáo cần được trình bày rõ ràng, súc tích. Các hình ảnh minh họa phải sắc nét và có chú thích đầy đủ. Đây là cơ sở quan trọng để giảng viên đánh giá mức độ hiểu của bạn về FP – điều mà chỉ nhìn vào code không thể thấy hết.

VII. Chính sách sử dụng công cụ AI (AI Usage Policy)

Trong khoá học này, các bạn **được phép sử dụng các công cụ AI** (ChatGPT, Copilot, Claude, Gemini, ...) như công cụ hỗ trợ học tập. Tuy nhiên, việc sử dụng AI cần tuân thủ các nguyên tắc sau.

VII.1 Nguyên tắc chung

- AI chỉ nên được sử dụng như **công cụ hỗ trợ học tập**, không thay thế quá trình suy nghĩ của bạn,
- Sinh viên cần **hiểu rõ mã nguồn** mình nộp,
- Sinh viên phải có khả năng **giải thích cách hoạt động của chương trình**.

Trong quá trình chấm hoặc vấn đáp, nếu bạn không thể giải thích mã nguồn của mình, bài làm có thể bị đánh giá lại. Đặc biệt với Level 3, AI có thể tạo ra code FP rất “đẹp” nhưng nếu bạn không nắm được *vì sao dùng compose thay vì pipe, hay vì sao cần curry*, sẽ rất khó qua được vấn đáp.

VII.2 Yêu cầu khai báo

Nếu các bạn sử dụng AI trong quá trình làm bài, cần khai báo rõ trong báo cáo. Việc khai báo này giúp đảm bảo tính minh bạch trong học tập.

Nếu sử dụng AI, cần khai báo rõ trong báo cáo:

- Tên công cụ AI,
- Mục đích sử dụng (chi tiết),
- Mức độ sử dụng (ước lượng).

Ví dụ:

AI Tool: Claude

Purpose: hỏi ý nghĩa cú pháp `std::ranges::views::filter`; gợi ý cách cài đặt curry trong C++; review chung mình quy nạp.

Estimated usage: ~25%

VII.3 Những việc không được phép

Sinh viên **không được phép**:

- Sao chép toàn bộ mã nguồn AI sinh ra mà không hiểu nội dung,
- Nộp bài giống nhau giữa các sinh viên,

- Sử dụng AI để qua hệ thống kiểm tra mà không thực sự làm bài.

Nếu phát hiện hành vi vi phạm học thuật, bài làm có thể bị xử lý theo quy định của môn học và của nhà trường.

VII.4 Mục tiêu của chính sách

Mục tiêu của chính sách này không phải là cấm AI, mà là khuyến khích các bạn:

- Sử dụng AI có trách nhiệm,
- Hiểu rõ mã nguồn của mình,
- Phát triển tư duy lập trình hàm thực sự – một trong những tư duy “khó học” nhất nếu chỉ đọc code.

Đội ngũ trợ giảng thực hành môn học chúng mình khuyến khích các bạn làm bài tập này bằng đúng khả năng của bản thân. Dù kết quả có như thế nào thì đây cũng là một trải nghiệm đáng được dành thời gian và tâm sức của mình vào.

Chúc các bạn có một hành trình thử sức thú vị với FP – mẫu hình lập trình khác biệt nhất trong ba mẫu hình của khoá học!

VIII. Nộp bài (Submission)

Sau đây là hướng dẫn chuẩn bị cho phần nộp bài. Yêu cầu các bạn đọc kỹ để nộp bài đúng yêu cầu.

VIII.1 Các thành phần cần nộp

Bài nộp cần bao gồm các thành phần sau:

- **Mã nguồn chương trình** (toàn bộ thư mục src/),
- **Tài nguyên trò chơi** (các file đa phương tiện .png, .ttf, .jpg, ...) – nếu cài đặt SDL
- **Báo cáo PDF** (file .pdf),
- **Thư mục kết quả thực nghiệm** (thư mục results/),
- **Thư mục trích dẫn log** (thư mục logs/) – chứa log đo thời gian và kết quả *Mức 3 nhánh B* (nếu có)

Toàn bộ các file cần được đặt trong một thư mục duy nhất trước khi nén.

VIII.2 Cấu trúc thư mục

```
project_level-3/
├── src/
│   ├── main.cpp
│   ├── core/
│   ├── shell/
│   ├── interface/
│   ├── terminal/
│   ├── (sdl/ -- tùy chọn)
│   └── utils/
├── results/
│   ├── grader-pass.png
│   ├── log-sample.txt
│   ├── (mục 3B) benchmark-chart.png
│   ├── (mục 3B) benchmark-table.csv
│   └── ...
└── logs/
    └── log_default.txt
```

```

|   |   | (muc 3B) log_seq_5x5.txt
|   |   | (muc 3B) log_par_5x5.txt
|   |   | ...
|   |   |
|   |   | CMakeLists.txt
|   |   | README.md (optional)
|   |   | report_level-3_<HoVaTen>_<MSSV>.pdf

```

Trong đó:

- report_level-3_<HoVaTen>_<MSSV>.pdf là file báo cáo PDF,
- src/ chứa toàn bộ mã nguồn của bài tập lớn sau khi được hoàn thiện,
- assets/ chứa toàn bộ tài nguyên của trò chơi (nếu có cài SDL),
- logs/ chứa file log được tạo ra khi chạy chương trình (minh chứng inject Logger vào Engine, ...); hoặc kết quả mức 3B *phần kết quả được dùng trong báo cáo nên được để vào thư mục này (minh chứng)*
- results/ chứa các ảnh chụp màn hình trong quá trình tương tác giao diện game; hoặc hình ảnh biểu đồ so sánh và kết quả so sánh (.csv) của mức 3B *các ảnh, kết quả dùng trong báo cáo nên được để vào thư mục này (minh chứng)*
- Một số file hỗ trợ như CMakeLists.txt, ...các file cần sử dụng trong quá trình biên dịch chương trình (nếu cần thiết)
- (tùy chọn) README.txt mô tả ngắn gọn cách biên dịch và chạy chương trình

VIII.3 Định dạng nộp bài

Thư mục bài làm cần được nén thành một file:

project_level-3_<HoVaTen>_<mssv>.zip

Ví dụ:

project_level-3_NguyenVanA_2502XXXX.zip

Chỉ nộp **một file nén duy nhất**.

VIII.4 Thời hạn nộp bài

Thời hạn được thông báo trên hệ thống của môn học. Vì Level 3 là *bonus*, các bạn không cần áp lực bằng Level 1 và 2 – nhưng cũng không nên để quá sát hạn.

VIII.5 Lưu ý

- Đảm bảo chương trình biên dịch và chạy được; nếu có cách biên dịch khác, ghi rõ trong README,
- Kiểm tra file nén trước khi nộp,
- Không nộp file object, file tạm, thư mục build/.